

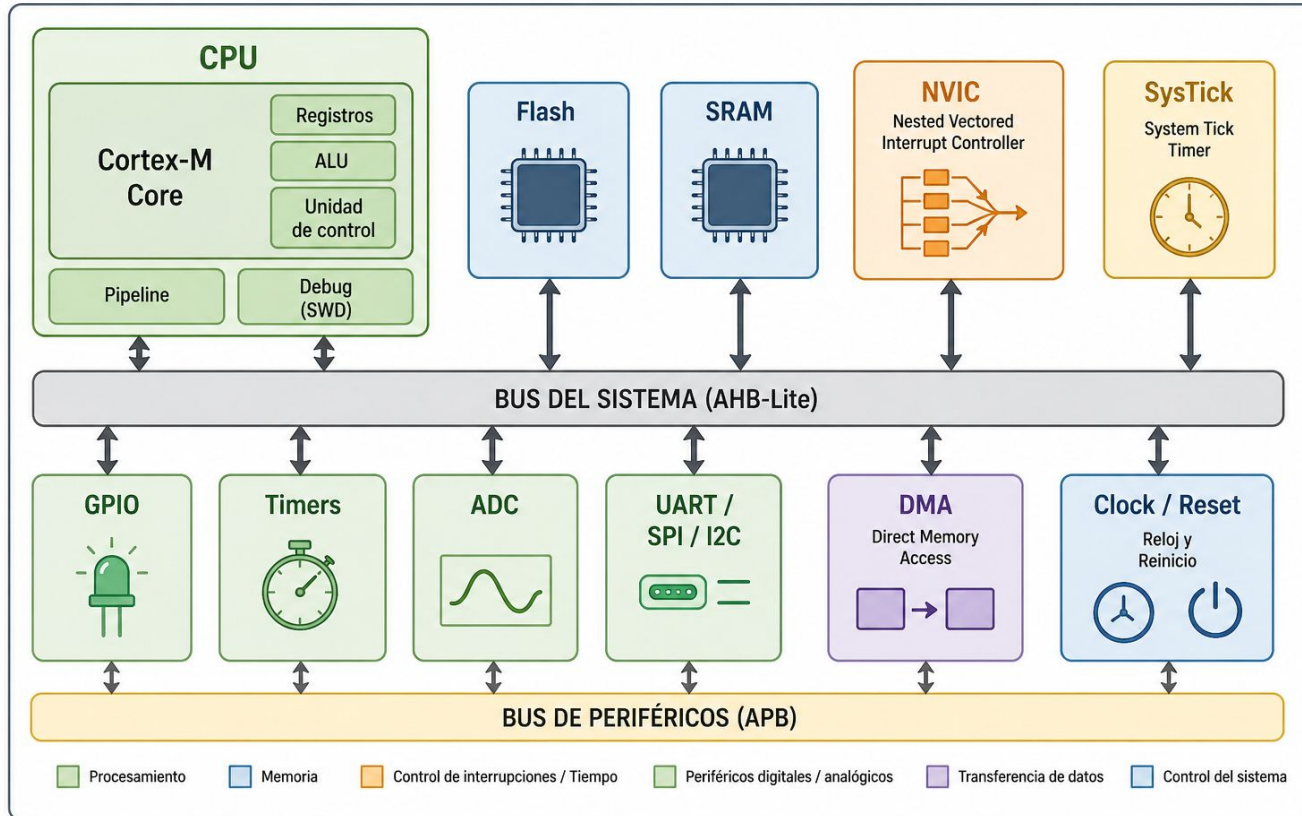
ADC: conversor analógico - digital

Clase 5 - 12/05/2026

Objetivo de la clase

- Explicar qué problema resuelve un ADC dentro de un sistema embebido.
- Interpretar una lectura digital de 12 bits y convertirla a tensión.
- Configurar un canal ADC en STM32CubeIDE para la NUCLEO-F446RE.
- Implementar una lectura ADC por polling usando HAL.
- Identificar limitaciones eléctricas y errores comunes al medir señales analógicas.
- Relacionar el ADC con sensores reales, acondicionamiento de señal y firmware embebido.

Conversor Analógico-Digital ADC en STM32



¿Por qué necesitamos un ADC?

Ejemplos de señales analógicas



Ejemplo de sensores analógicos

MP5010DP

Sensor de presión



- Rango: 0 a 10 kPa
- Salida analógica ratiométrica
- Entrega una tensión proporcional a la presión aplicada

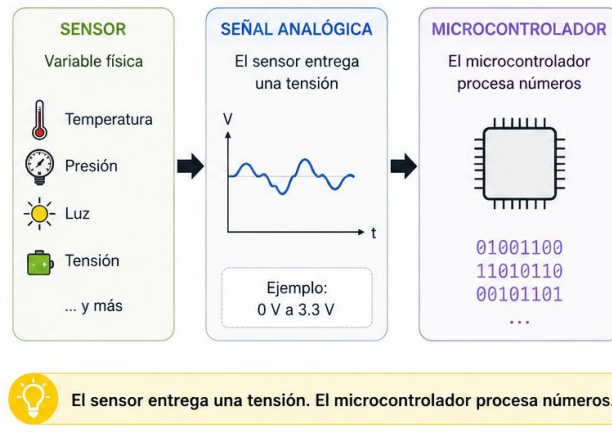
LM35

Sensor de temperatura



- Rango: -55 °C a +150 °C
- Salida: 10 mV / °C
- Entrega una tensión proporcional a la temperatura

¿Cómo funciona?



Cadena típica de adquisición analógica

Sensor

→ acondicionamiento de señal

→ entrada analógica del microcontrolador

→ ADC

→ valor digital

→ procesamiento en firmware

→ decisión / comunicación / control

Conversión analógico-digital

Una señal analógica es continua. El ADC entrega un número discreto.

Concepto	Expresión
ADC de N bits	niveles = 2^N
ADC de 12 bits	$2^{12} = 4096$ niveles
Rango de entrada	$0 \text{ V} \leq V_{IN} \leq V_{REF}$
Referencia típica	$V_{REF} \approx 3.3 \text{ V}$
Valor mínimo	ADC = 0 $\rightarrow$ 0 V
Valor máximo	ADC = 4095 $\rightarrow$ 3.3 V

Ejemplo aplicado a la placa STM32 NUCLEO-F446RE con ADC de 12 bits y referencia aproximada de 3.3 V.

Resolución, granularidad y tamaño del paso

Cantidad de niveles digitales que puede representar el ADC.

$$\text{niveles} = 2^N$$

$$\text{LSB} = \frac{V_{REF}}{2^N}$$

Ejemplo con $V_{REF} = 3.3 \text{ V}$

Resolución	Niveles	Salida	Paso aprox.
10 bits	1024	0 a 1023	3.22 mV
12 bits	4096	0 a 4095	0.805 mV
16 bits	65536	0 a 65535	0.050 mV

Granularidad: tamaño del escalón mínimo distinguible.
Más bits → menor paso → mayor granularidad.

Importante: mayor resolución no garantiza mayor precisión.

ADC en STM32F446RE

Características relevantes:

- Tres ADC de 12 bits.
- Varios canales multiplexados.
- Modos single, continuous, scan, discontinuous.
- Lectura por polling, interrupción o DMA.
- Posibilidad de disparo por software o por temporizador.
- Aplicaciones: sensores, baterías, control, adquisición de datos.

Pines analógicos en NUCLEO-F446RE

Pin Arduino	Pin STM32	Canal
A0	PA0	ADC123_IN0
A1	PA1	ADC123_IN1
A2	PA4	ADC12_IN4
A3	PB0	ADC12_IN8
A4	PC1 / PB9	ADC123_IN11 / I2C1_SDA
A5	PC0 / PB8	ADC123_IN10 / I2C1_SCL

Advertencia eléctrica

- **Regla de seguridad**
 - No aplicar 5 V a una entrada analógica.
 - Rango de trabajo para esta clase:

$$0\text{ V} \leq V_{IN} \leq 3.3\text{ V}$$

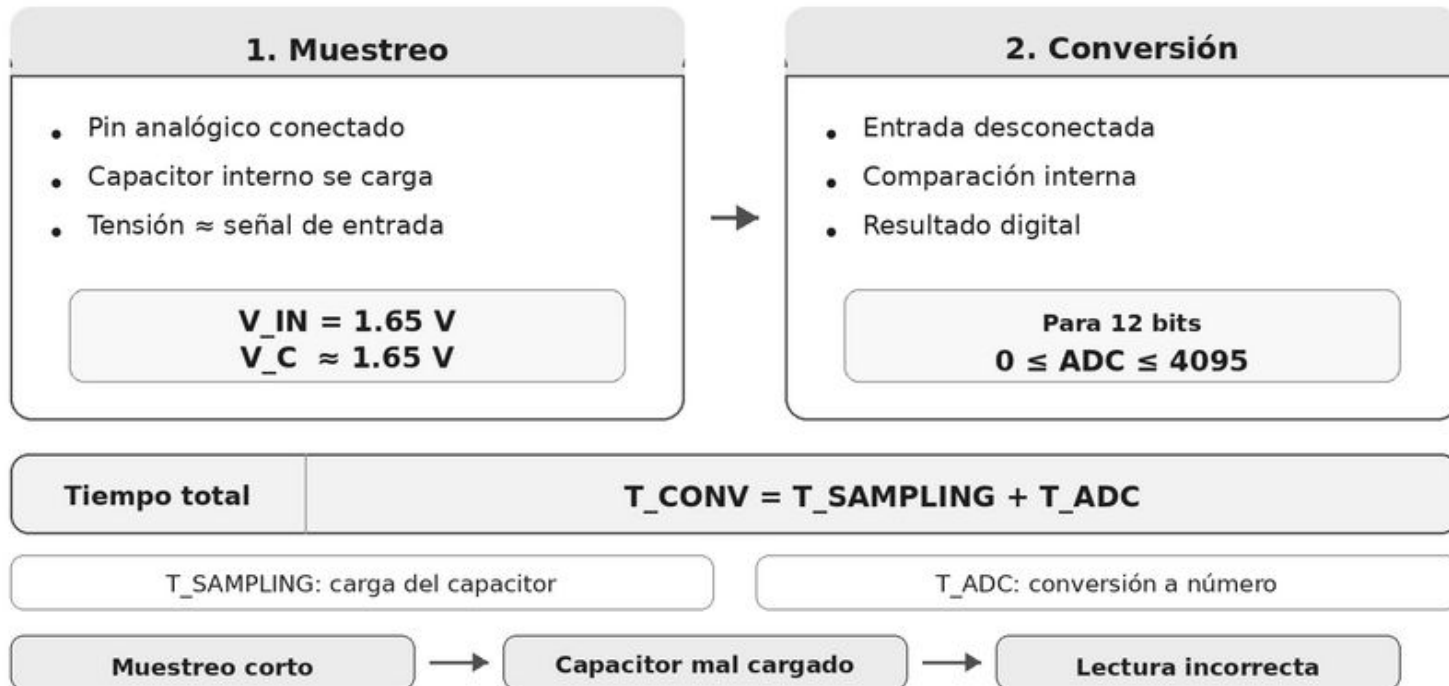
- **Conexión del potenciómetro:**
 - a. Extremo 1 $\rightarrow$ 3,3 V
 - b. Extremo 2 $\rightarrow$ GND
 - c. Cursor $\rightarrow$ A2 / PA4

Potenciómetro sugerido: 10 k Ω .

Proceso interno: muestreo y conversión

Conversión ADC: muestreo + conversión

La conversión no ocurre de forma instantánea.



Tiempo de muestreo y velocidad de adquisición

Configuración	Tiempo de muestreo	Tiempo de conversión	Tiempo total por muestra	Muestras por segundo
Rápida	1 μ s	1 μ s	2 μ s	500000 muestras/s
Más estable	9 μ s	1 μ s	10 μ s	100000 muestras/s

Aumentar el tiempo de muestreo puede mejorar la calidad de la lectura, pero reduce la cantidad máxima de muestras por segundo.

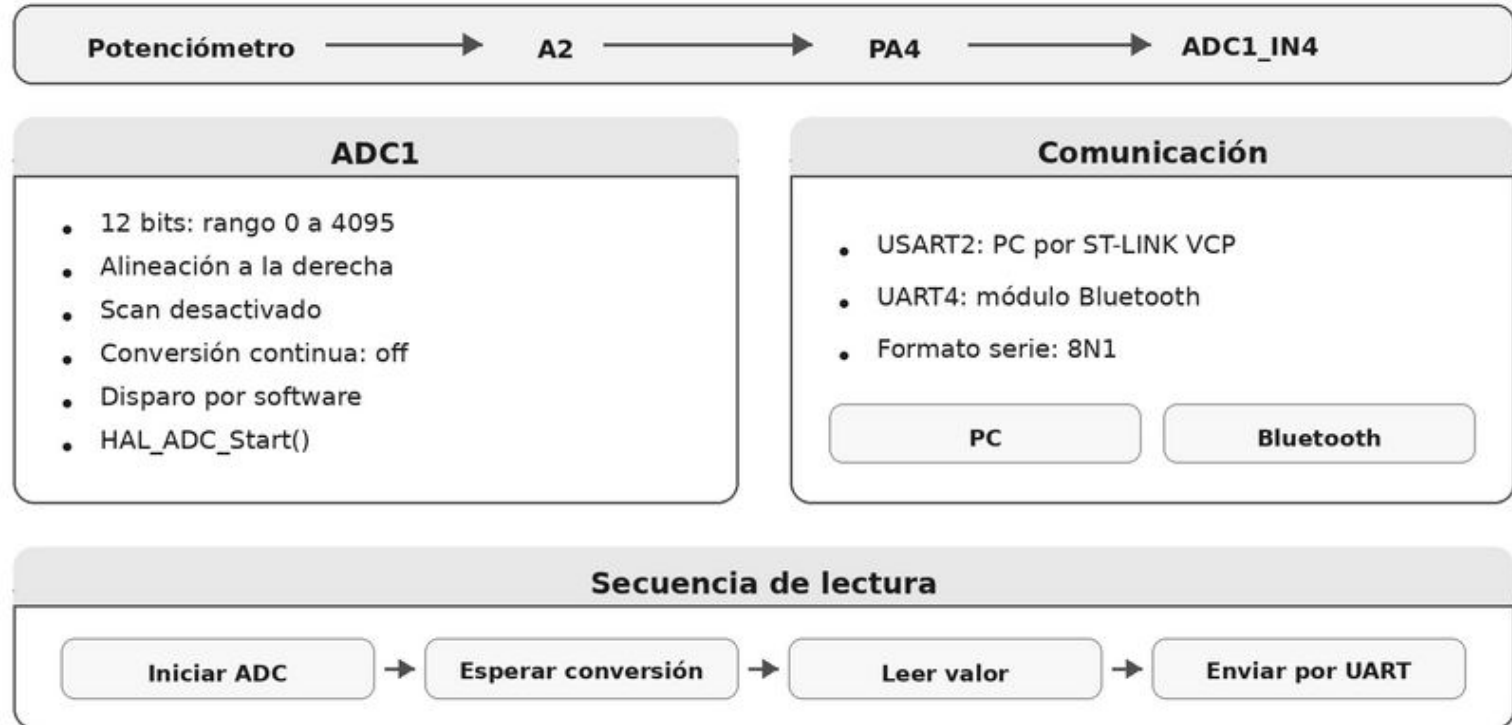
Ganamos estabilidad, pero perdemos velocidad de adquisición.

Modos de uso del ADC

Formas habituales de usar el ADC

1. **Single conversion**
 - Una conversión por solicitud.
2. **Continuous conversion**
 - Conversiones repetidas automáticamente.
3. **Scan mode**
 - Secuencia de varios canales.
4. **Polling**
 - El programa espera el fin de conversión.
5. **Interrupt**
 - El ADC avisa al finalizar.
6. **DMA**
 - Los datos pasan a memoria sin intervención constante de CPU.

Configuración del ADC en STM32CubeIDE



Lectura bajo control del firmware: una conversión por orden del programa.

Flujo del firmware

Secuencia de lectura por polling

1. Iniciar conversión.
2. Esperar fin de conversión.
3. Leer valor digital.
4. Detener conversión.
5. Convertir a tensión.
6. Enviar por UART.
7. Repetir.

```
HAL_ADC_Start(&hadc1);  
HAL_ADC_PollForConversion(&hadc1,  
HAL_MAX_DELAY);  
adc_raw = HAL_ADC_GetValue(&hadc1);  
HAL_ADC_Stop(&hadc1);  
  
voltage = (3.3f * adc_raw) / 4095.0f;
```

Código base para el laboratorio

```
uint32_t adc_raw = 0;
float voltage = 0.0f;

while (1)
{
    HAL_ADC_Start(&hadc1);

    if (HAL_ADC_PollForConversion(&hadc1, 100) == HAL_OK)
    {
        adc_raw = HAL_ADC_GetValue(&hadc1);
        voltage = (3.3f * adc_raw) / 4095.0f;
        printf("ADC: %lu | V: %.3f\r\n", adc_raw, voltage);
    }

    HAL_ADC_Stop(&hadc1);
    HAL_Delay(200);
}
```


Laboratorio guiado

Ejercicio 1 — Lectura de potenciómetro

Objetivo:

- Leer una tensión variable en A2.
- Mostrar valor ADC y tensión por UART.

Pasos:

1. Conectar potenciómetro.
2. Configurar ADC1_IN4.
3. Activar USART2.
4. Programar la placa.
5. Abrir terminal serie.
6. Girar el potenciómetro.
7. Observar valores entre 0 y 4095.

Laboratorio guiado

Ejercicio 2 – Umbral

Usar la lectura ADC para tomar una decisión:

```
if (voltage > 2.0f)
{
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);
}
else
{
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET);
}
```

LD2 en NUCLEO-F446RE:

- Pin PA5.
- LED verde de usuario.

Precisión, ruido y estabilidad

Causas frecuentes:

- Ruido en la señal.
- Ruido en la referencia.
- Cables largos.
- Mala conexión a GND.
- Fuente con alta impedancia.
- Tiempo de muestreo demasiado corto.
- Sensor sin acondicionamiento.
- Promediado inexistente.

Mejoras posibles:

- Promediar muestras.
- Usar filtros RC.
- Mejorar GND y cableado.
- Aumentar sampling time.
- Separar señales analógicas y digitales.
- Usar referencia estable.

Promediado simple: N muestras

```
uint32_t adc_sum = 0;

uint32_t adc_avg = 0;

for (int i = 0; i < 16; i++)
{
    HAL_ADC_Start(&hadc1);

    HAL_ADC_PollForConversion(&hadc1, 100);

    adc_sum += HAL_ADC_GetValue(&hadc1);

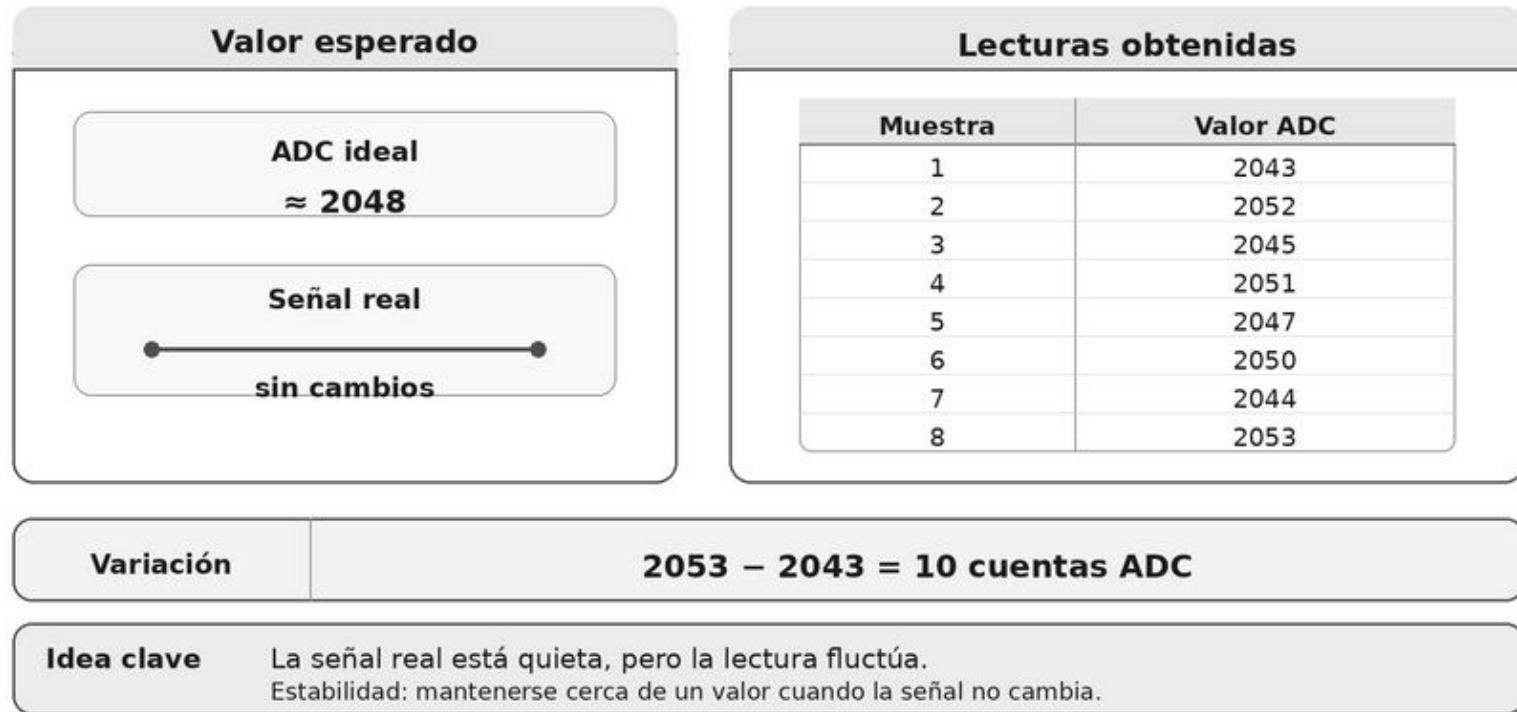
    HAL_ADC_Stop(&hadc1);
}

adc_avg = adc_sum / 16;

voltage = (3.3f * adc_avg) / 4095.0f;
```

Estabilidad de la lectura ADC

Caso: el potenciómetro está quieto.



Promedio simple

Promediado de muestras ADC

Objetivo: obtener una medición puntual más representativa.

En lugar de usar una sola muestra, se promedian varias mediciones.

Muestras ADC

2043

2052

2045

2051

2047

2050

2044

2053

Cálculo del promedio

$$\text{Promedio} = (2043 + 2052 + 2045 + 2051 + 2047 + 2050 + 2044 + 2053) / 8$$

$$\text{Promedio} = 2048.125$$

Una sola muestra
puede estar afectada por ruido



Promedio de muestras
representa mejor el valor real

Promedio móvil

Promedio móvil de muestras ADC

Objetivo: suavizar una medición continua.

Se promedian siempre las últimas N muestras. Ejemplo: ventana de 4 muestras.

Ejemplo con ventana de 4 muestras

Muestra actual	Últimas 4 muestras	Promedio móvil
4	2043, 2052, 2045, 2051	2047.75
5	2052, 2045, 2051, 2047	2048.75
6	2045, 2051, 2047, 2050	2048.25
7	2051, 2047, 2050, 2044	2048.00
8	2047, 2050, 2044, 2053	2048.50

Nueva muestra entra



Muestra más antigua sale



Promedio actualizado

Reduce fluctuaciones rápidas

lectura más suave

Puede introducir retardo

respuesta más lenta

Comparación

Tres conceptos relacionados, pero con funciones distintas.

Concepto	Qué es	Para qué sirve	Cuándo usarlo	Limitación
Estabilidad	Característica de la medición	Evaluar cuánto fluctúa la lectura	Siempre que se mida una señal analógica	No es una técnica de corrección
Promedio simple	Técnica por bloque	Obtener una medición puntual más representativa	Señales lentas o mediciones aisladas	Hay que esperar varias muestras
Promedio móvil	Filtro digital continuo	Suavizar la lectura en tiempo real	Monitoreo continuo de señales lentas o moderadas	Introduce retardo ante cambios rápidos

Conclusión

Estabilidad

lo que queremos mejorar

Promedio simple

técnica para una medición puntual

Promedio móvil

técnica para suavizar una medición continua

Errores frecuentes

Errores típicos al usar ADC en STM32

1. Conectar 5 V a una entrada analógica.
2. No unir GND del sensor con GND de la placa.
3. Configurar el pin equivocado.
4. Confundir A0 con PA0.
5. Usar una referencia inestable.
6. Leer sin esperar fin de conversión.
7. No considerar impedancia de fuente.
8. Usar cables largos sin filtrado.
9. Interpretar 4095 como “4095 voltios”.
10. Confundir resolución con precisión.

Diferencia entre ADC y canales analógicos

Pin placa	Pin STM32	Función ADC
A0	PA0	ADC123_IN0
A1	PA1	ADC123_IN1
A2	PA4	ADC12_IN4
A3	PB0	ADC12_IN8
A4	PC1	ADC123_IN11
A5	PC0	ADC123_IN10

Cierre conceptual

El ADC permite que el firmware observe el mundo físico.

Pero una buena medición requiere:

- Rango eléctrico correcto.
- Referencia estable.
- Pin bien configurado.
- Tiempo de muestreo adecuado.
- Lectura correcta desde firmware.
- Interpretación física del dato.
- Criterio de diseño.

Actividad opcional recomendable

Modificar el programa para que:

1. Lea el potenciómetro conectado a A2.
2. Calcule la tensión.
3. Encienda LD2 si la tensión supera 2.0 V.
4. Envíe por UART:
 - valor ADC crudo,
 - tensión calculada,
 - estado del LED.

Preguntas de reflexión

1. ¿Qué valor ADC se obtiene aproximadamente para 1.65 V?
2. ¿Qué ocurre si el potenciómetro no comparte GND con la placa?
3. ¿Por qué no debe conectarse una salida de 5 V directamente al ADC?
4. ¿Qué diferencia hay entre resolución y precisión?
5. ¿Cuándo conviene usar DMA en lugar de polling?